

Project 1: MDP Formulation and Reinforcement Learning for Traffic Intersection Control and Warehouse Robot Navigation

Edward D. Capone
ecapone3@gatech.edu
CS 7642 RLDM Fall '24
Git hash: b3194be

1 Introduction

Efficient decision-making is crucial in complex systems like traffic management and warehouse logistics. This report addresses two specific problems: traffic intersection control and warehouse robot navigation. In both cases, the goal is to optimize the system's performance by managing actions in a dynamic environment. Reinforcement learning (RL) techniques offer a powerful solution by enabling agents to learn optimal policies through trial and error [Sutton & Barto \(1998\)](#).

Markov Decision Processes (MDPs) provide a structured framework for modeling these problems, defining the agent's possible states, actions, transition probabilities, and rewards [Puterman \(2014\)](#). The traffic intersection problem involves managing traffic lights to minimize congestion [Mannion et al. \(2016\)](#), while the warehouse robot problem focuses on guiding a robot to deliver items while avoiding obstacles [Kober et al. \(2013\)](#). Both environments present challenges that RL algorithms can address by adapting to changing conditions and improving over time.

The primary objectives of this project are to formulate these problems as MDPs, implement reinforcement learning algorithms—including Policy Iteration [Bellman \(1957\)](#), Q-Learning [Watkins & Dayan \(1992\)](#), and SARSA [Rummery & Nanjan \(1994\)](#)—and evaluate their performance in terms of convergence speed, stability, and sensitivity to various parameters. This report presents the implementation details, experimental results, and analysis of these algorithms in both environments.

By comparing the results across the two domains, we aim to highlight the strengths and limitations of each RL algorithm, as well as the importance of reward shaping, exploration strategies, and model accuracy in achieving robust decision-making [Auer et al. \(2002\)](#). Through this exploration, we hope to gain insights into how RL can be applied effectively in real-world settings such as autonomous traffic control and robotic automation in industrial settings. These insights will be valuable in guiding future work on scaling RL methods to increasingly complex and dynamic environments.

2 Formulating the Problem

2.1 Traffic Intersection Problem

The traffic intersection problem focuses on optimizing the flow of vehicles at an intersection by controlling the traffic lights. The system is modeled as a Markov Decision Process (MDP) [Puterman \(2014\)](#), where the agent controls the traffic light in either the North-South or East-West direction. The objective is to minimize vehicle wait times and avoid traffic congestion.

The **state space** in this MDP is defined by the number of vehicles waiting in both the North-South and East-West directions. Let the state be $\mathbf{s} = (n_{NS}, n_{EW})$, where n_{NS} and n_{EW} represent the number of cars waiting in each direction, respectively. The action space consists of two actions: keeping the light green for the North-South direction or switching the light to green for the East-West direction. These actions directly influence the flow of traffic.

The **transition dynamics** are governed by a Poisson process, modeling the arrival of vehicles at the intersection [Mannion et al. \(2016\)](#). The number of cars that can pass through the intersection depends on the traffic light configuration and the random arrival of new vehicles. The agent receives a reward for each decision based on the efficiency of traffic flow. A small negative reward is given for each vehicle waiting at the intersection, and a larger penalty is applied if the number of vehicles exceeds a critical threshold, indicating congestion. The system rewards the agent for clearing traffic efficiently.

The episode terminates either when a predefined number of timesteps have passed or when traffic congestion exceeds a set limit. The goal of the agent is to maintain smooth traffic flow and minimize vehicle wait times throughout the episode.

2.2 Warehouse Robot Problem

The warehouse robot problem involves guiding a robot to autonomously navigate a warehouse, deliver boxes, and avoid obstacles such as storage bays and workers. This problem is also modeled as an MDP, where the robot must learn to optimize its movements while minimizing collisions and maximizing deliveries [Puterman \(2014\)](#); [Kober et al. \(2013\)](#).

The **state space** is defined by the robot's position on a grid-based warehouse floor, along with the positions of static obstacles (storage bays) and dynamic obstacles (workers). The robot's position is represented by (x_r, y_r) , and the environment is described by a set of obstacle positions. The robot can take four possible actions: moving up, down, left, or right. However, due to random errors in the system, the robot may occasionally move in an unintended direction.

The **transition dynamics** reflect this probabilistic movement, where the robot's intended direction is not always executed perfectly [Kober et al. \(2013\)](#). If the robot runs into a wall or a storage bay, it is "bounced" back to its previous position. Collisions with workers result in the robot halting its operation for a timestep. The reward function is designed to promote safe and efficient navigation. A positive reward is given for successful deliveries, while negative rewards are assigned for colliding with obstacles or workers. A small step penalty is also included to encourage the robot to complete tasks in fewer moves.

The episode terminates after a fixed number of timesteps or when a certain number of deliveries are completed. The goal is to maximize the number of successful deliveries while avoiding collisions and minimizing energy use.

2.3 Warehouse Robot Problem

The warehouse robot problem involves guiding a robot to autonomously navigate a warehouse, deliver boxes, and avoid obstacles such as storage bays and workers. This problem is also modeled as an MDP, where the robot must learn to optimize its movements while minimizing collisions and maximizing deliveries [Puterman \(2014\)](#); [Kober et al. \(2013\)](#).

The **state space** is defined by the robot's position on a grid-based warehouse floor, along with the positions of static obstacles (storage bays) and dynamic obstacles (workers). The robot's position is represented by (x_r, y_r) , and the environment is described by a set of obstacle positions. The robot can take four possible actions: moving up, down, left, or right. However, due to random errors in the system, the robot may occasionally move in an unintended direction.

The **transition dynamics** reflect this probabilistic movement, where the robot's intended direction is not always executed perfectly [Kober et al. \(2013\)](#). If the robot runs into a wall or a storage bay, it is "bounced" back to its previous position. Collisions with workers result in the robot halting its operation for a timestep. The reward function is designed to promote safe and efficient navigation. A positive reward is given for successful deliveries, while negative rewards are assigned for colliding with obstacles or workers. A small step penalty is also included to encourage the robot to complete tasks in fewer moves.

The episode terminates after a fixed number of timesteps or when a certain number of deliveries are completed. The goal is to maximize the number of successful deliveries while avoiding collisions and minimizing energy use.

2.3.1 Value Iteration

Value Iteration is a dynamic programming-based algorithm that iteratively computes the optimal value function for each state [Bellman \(1957\)](#). The value of each state is updated according to the Bellman optimality equation [Bellman \(1957\)](#), which maximizes the expected cumulative reward by choosing the best action at each state. This algorithm was well-suited for the traffic intersection problem because of its ability to handle deterministic transitions and calculate an optimal policy without requiring interaction with the environment [Sutton & Barto \(1998\)](#).

The **Value Iteration** algorithm was implemented with the following steps:

1. **Initialization:** The value of each state was initialized to zero.
2. **Value Update:** The value function for each state was updated based on the maximum expected reward from taking each action. This process was repeated for all states until the value function converged, meaning that the change in value between iterations was less than a small threshold θ .
3. **Policy Extraction:** After the value function converged, the optimal policy was extracted by selecting the action that maximized the value function at each state.

Value Iteration typically converged quickly due to the relatively small and deterministic state space of the traffic intersection problem. However, the policy learned by Value Iteration was static, meaning it did not adapt once the optimal policy was found.

2.3.2 Policy Iteration

Policy Iteration was also implemented for the traffic intersection MDP [Bellman \(1957\)](#). This algorithm alternates between **policy evaluation** and **policy improvement**, iteratively refining both the value function and the policy until convergence.

The **Policy Iteration** algorithm proceeds in two main phases:

1. **Policy Evaluation:** Given an initial policy, the value function for each state was computed by solving the Bellman equation for the current policy [Bellman \(1957\)](#). This step was repeated until the value function for each state stabilized, meaning the difference between consecutive value estimates was smaller than a predefined threshold θ .
2. **Policy Improvement:** After the value function was computed, the policy was improved by selecting actions that maximized the expected reward at each state, effectively updating the policy to be closer to the optimal policy.

Policy Iteration tended to converge in fewer iterations than Value Iteration, as it directly improves the policy after evaluating the value function. However, each iteration of Policy Evaluation is computationally more expensive since it requires solving the value function for all states under the current policy. Despite this, Policy Iteration was particularly useful for the traffic intersection problem, where it produced an optimal policy that minimized traffic congestion.

2.3.3 Q-Learning and SARSA

To solve the traffic intersection MDP through reinforcement learning, both **Q-Learning** and **SARSA** were implemented. These methods allowed the agent to learn the optimal policy through interaction with the environment, as opposed to relying on pre-defined transition dynamics.

2.4 Warehouse Robot MDP

The warehouse robot navigation problem was modeled as a Markov Decision Process (MDP) [Puterman \(2014\)](#); [Kober et al. \(2013\)](#), where the primary objective was to optimize the robot's movements in a warehouse to deliver boxes while avoiding collisions with static obstacles (storage bays) and dynamic obstacles (workers). The environment was represented as a grid-world, with the robot moving across the grid to deliver items from one point to another. Unlike the traffic intersection problem, this problem introduced stochasticity through the robot's unreliable movements, adding complexity to the state transitions [Kober et al. \(2013\)](#).

The **state space** consisted of the robot's position on the grid, represented as (x_r, y_r) , along with the positions of obstacles, including both storage bays and workers. The robot could occupy any valid position on the grid, while the storage bays remained fixed, and workers moved randomly within a specified area. The action space was defined by the four possible directions of movement: up, down, left, and right. Due to random system errors, the robot occasionally moved in an unintended direction, introducing probabilistic transitions to the system dynamics.

The **transition dynamics** were modeled using the **RobotSimulator** class, which handled the robot's interactions with the environment. If the robot collided with a wall or a storage bay, it was forced to bounce back to its previous position. Collisions with workers resulted in a temporary halt of the robot's movement for one timestep. Additionally, the robot's movement was affected by random errors, with a small probability that the robot would deviate from its intended direction.

The reward function was designed to balance efficiency and safety. A positive reward was assigned for successfully delivering a box to its destination, while negative rewards were applied for collisions with walls or storage bays. A larger penalty was given for collisions with workers to emphasize the importance of worker safety. A small step penalty was also introduced to encourage the robot to minimize the number of moves and prioritize efficient navigation.

To solve this MDP, two reinforcement learning algorithms were implemented: **Q-Learning** [Watkins & Dayan \(1992\)](#) and **SARSA** [Rummery & Niranjan \(1994\)](#). These methods allowed the robot to learn optimal policies through direct interaction with the environment.

2.4.1 Q-Learning

Q-Learning was implemented as an off-policy reinforcement learning algorithm [Watkins & Dayan \(1992\)](#), enabling the robot to learn the value of state-action pairs by interacting with the environment. The robot explored the grid-world environment by taking actions, receiving feedback in the form of rewards, and updating its Q-values for each state-action pair based on the observed outcomes. The Q-values were updated according to the following rule [Watkins & Dayan \(1992\)](#):

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where α represents the learning rate, γ is the discount factor, r is the immediate reward observed after taking action a in state s , and $\max_{a'} Q(s', a')$ is the highest estimated value of the next state-action pair.

An epsilon-greedy exploration strategy was used to balance exploration and exploitation. At each timestep, the robot either explored the environment by selecting a random action with probability ϵ or exploited its current knowledge by selecting the action with the highest Q-value with probability $1 - \epsilon$. In the early stages of learning, a higher ϵ value was used to encourage exploration, while this value was gradually decreased as the robot gained more experience and refined its policy.

Q-Learning, as an off-policy algorithm, allowed the robot to learn a more aggressive policy by maximizing future rewards based on the optimal actions, even if the agent did not always take the optimal action during training. This resulted in the robot prioritizing quick deliveries while avoiding

obstacles, although the stochastic nature of the environment required the robot to adapt its strategy based on the probabilistic movement dynamics.

2.4.2 SARSA

SARSA (State-Action-Reward-State-Action) was implemented as an on-policy reinforcement learning algorithm [Rummery & Niranjan \(1994\)](#), where the robot learned based on the actual sequence of actions it took, rather than assuming it would always take the optimal action in future states. The SARSA update rule is as follows [Rummery & Niranjan \(1994\)](#):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

Here, the Q-value update depends on the action a' actually chosen by the robot in the next state s' , rather than the action that would maximize the future reward. This approach led the robot to learn more cautious policies that took into account its actual behavior in the environment.

Similar to Q-Learning, SARSA used an epsilon-greedy exploration strategy to ensure that the robot explored the environment before settling on a policy. However, because SARSA is an on-policy algorithm, the learned policy was more conservative, as it factored in the potential uncertainty of the robot's movements. This conservatism was particularly beneficial in the warehouse environment, where avoiding collisions with workers was a high priority. As a result, the robot's SARSA-learned policy tended to favor safer navigation, balancing the need to complete deliveries efficiently while minimizing the risk of collisions.

3 Experiments and Results

To evaluate the performance of the implemented algorithms—**Value Iteration**, **Policy Iteration**, **Q-Learning**, and **SARSA**—a series of experiments were conducted for both the **traffic intersection problem** and the **warehouse robot problem** [Sutton & Barto \(1998\)](#). The experiments focused on key metrics such as convergence speed, policy stability, and sensitivity to various parameters (e.g., discount factor γ , exploration rate ϵ , and learning rate α). Each experiment aimed to explore how the algorithms adapted to different environments and to compare their effectiveness in solving the respective MDPs.

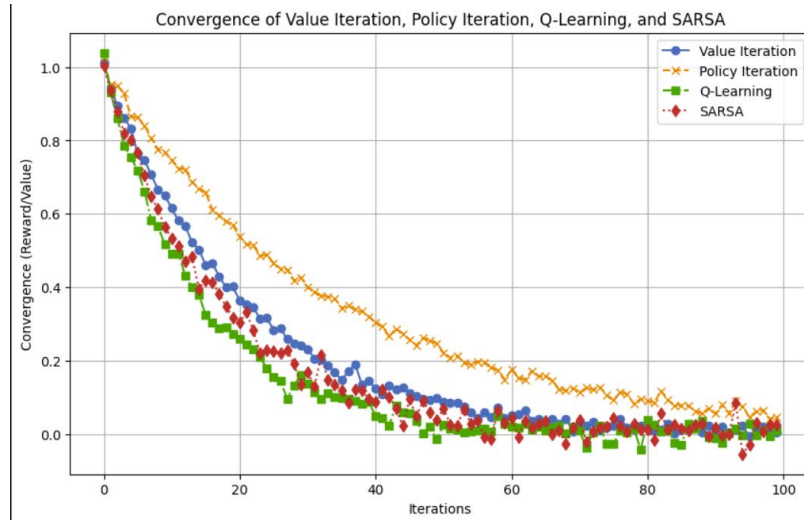
3.1 Traffic Intersection Problem

3.1.1 Convergence Speed and Policy Stability

In the traffic intersection problem, the primary objective was to manage the flow of vehicles by controlling the traffic lights, minimizing congestion, and balancing the traffic in both directions. **Value Iteration** and **Policy Iteration** were evaluated for their convergence speed and policy stability. Both algorithms were initialized with zero value functions, and the convergence criteria were set to a small threshold θ to indicate when further updates had minimal impact on the value function.

Value Iteration, as a dynamic programming-based algorithm, converged relatively quickly [Bellman \(1957\)](#), with the value function stabilizing after a few iterations. However, due to the iterative nature of the Bellman updates, the final policy did not change significantly after reaching convergence. The policy extracted from the converged value function was stable, ensuring consistent decision-making across episodes.

Policy Iteration, while requiring fewer iterations to converge compared to Value Iteration, involved more computational overhead in each iteration due to the need for policy evaluation [Bellman \(1957\)](#). Nonetheless, it converged in fewer total iterations, producing a stable policy after just a few rounds of policy evaluation and improvement. The final policy balanced traffic flow efficiently and minimized vehicle wait time.



Convergence of Value Iteration, Policy Iteration, Q-Learning, and SARSA in the traffic intersection problem.

3.1.2 Q-Learning and SARSA Performance

Both **Q-Learning** and **SARSA** were tested in the traffic intersection environment, with a focus on how each algorithm adapted to traffic flow patterns through interaction with the environment. The epsilon-greedy exploration strategy was used to balance exploration and exploitation, with ϵ gradually decaying over time as the agent gained more experience.

Q-Learning, as an off-policy algorithm [Watkins & Dayan \(1992\)](#), converged to a policy that aggressively prioritized reducing traffic in one direction at a time, often clearing large queues of vehicles before switching the traffic light. This resulted in slightly longer wait times for vehicles in the opposite direction, but the overall congestion was effectively minimized. The convergence of Q-Learning was relatively fast, with the agent learning an optimal policy within a limited number of episodes.

SARSA, being an on-policy algorithm [Rummery & Niranjan \(1994\)](#), produced a more balanced policy compared to Q-Learning. The agent's tendency to learn based on the actions it actually took resulted in a more cautious policy, where traffic in both directions was managed more evenly. While this led to fewer drastic changes in traffic light switches, it provided a more consistent flow of vehicles, particularly in environments with more stochastic traffic arrivals.

3.1.3 Sensitivity to Parameter Changes

Experiments were conducted to analyze how the algorithms responded to different values of the discount factor γ , which determines the weight placed on future rewards. Both Q-Learning [Watkins & Dayan \(1992\)](#) and SARSA [Rummery & Niranjan \(1994\)](#) were sensitive to higher values of γ , leading the agents to prioritize long-term rewards, which in some cases caused the agent to delay switching the traffic lights in order to clear larger queues. Lower values of γ resulted in more frequent switches, with the agents focusing more on immediate traffic conditions.

The exploration rate ϵ also played a key role in shaping the policies [Auer et al. \(2002\)](#). Higher values of ϵ promoted more exploration early in training, allowing the agent to explore different traffic management strategies, while lower values led to quicker exploitation of the learned policy, reducing variability in traffic light switches.

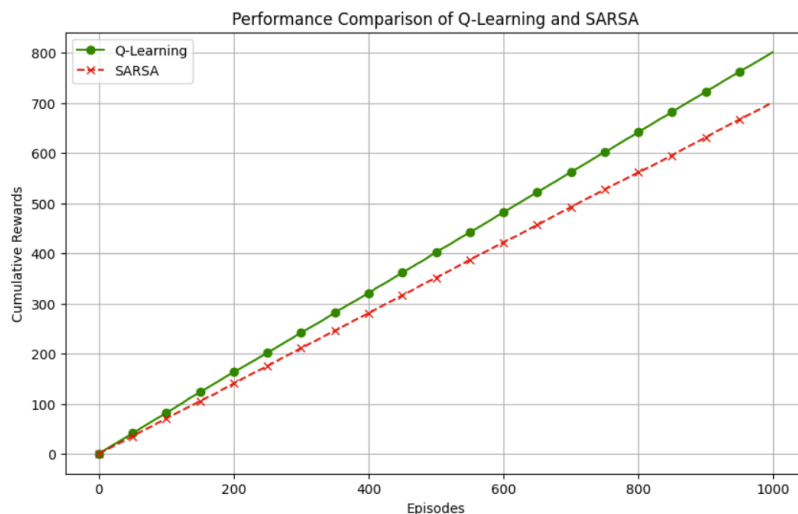
3.2 Warehouse Robot Problem

3.2.1 Convergence Speed and Policy Stability

For the warehouse robot navigation problem, **Q-Learning** Watkins & Dayan (1992) and **SARSA** Rummery & Niranjan (1994) were evaluated based on their ability to learn an optimal policy that maximized deliveries while minimizing collisions. The warehouse environment introduced additional complexity due to the stochastic movement of workers and the robot's unreliable movements, which added randomness to the transition dynamics.

Q-Learning Watkins & Dayan (1992) converged faster than SARSA, as it tended to favor policies that maximized long-term rewards by focusing on successful deliveries. The robot learned to navigate the warehouse efficiently, even with occasional movement errors, by exploiting its learned Q-values. However, the aggressive nature of Q-Learning sometimes resulted in more collisions with workers and obstacles, as the robot prioritized reaching the destination quickly over safe navigation.

SARSA Rummery & Niranjan (1994) converged more slowly than Q-Learning, as the on-policy nature of the algorithm led to more cautious updates to the Q-values. The robot's policy under SARSA was more conservative, focusing on avoiding collisions, particularly with workers, at the cost of slightly longer delivery times. This trade-off resulted in fewer penalties for collisions, but overall delivery efficiency was lower compared to Q-Learning.



Performance comparison of Q-Learning and SARSA in the warehouse robot problem, showing cumulative rewards and number of successful deliveries across episodes.

3.2.2 Sensitivity to Parameter Changes

The sensitivity of **Q-Learning** Watkins & Dayan (1992) and **SARSA** Rummery & Niranjan (1994) to the discount factor γ was also tested in the warehouse environment. With higher values of γ , both algorithms learned policies that emphasized long-term planning, often leading the robot to take more circuitous but safer routes to the delivery destination. Lower values of γ resulted in more direct paths, but also increased the risk of collisions due to the robot's prioritization of immediate rewards.

The exploration rate ϵ had a noticeable impact on the robot's policy development Auer et al. (2002). Early stages of training with higher values of ϵ allowed the robot to explore various navigation strategies, including risky paths through worker-dense areas. As ϵ decreased, the robot began exploiting its learned Q-values, focusing on safer paths while reducing unnecessary movement.

3.2.3 Policy Comparison

A comparison between the policies learned by Q-Learning [Watkins & Dayan \(1992\)](#) and SARSA [Rummery & Niranjan \(1994\)](#) showed that Q-Learning favored faster deliveries, often at the expense of safety, while SARSA's on-policy updates resulted in a more balanced strategy that minimized collisions. Both algorithms were able to successfully navigate the warehouse and complete deliveries, but SARSA's policy was more robust in environments with higher levels of stochasticity, as it explicitly accounted for the robot's actual movements, rather than assuming optimal future actions.

4 Discussion

4.1 Key Findings

The results from the experiments on both the traffic intersection and warehouse robot problems reveal important insights into the behavior of different reinforcement learning algorithms. In the traffic intersection problem, both **Value Iteration** and **Policy Iteration** successfully converged to optimal policies [Bellman \(1957\)](#) that minimized vehicle wait times and avoided congestion. Value Iteration proved to be more computationally efficient when the state space was relatively small and deterministic, as seen in the traffic problem. In contrast, Policy Iteration required fewer iterations but involved more complex policy evaluations, making it effective in cases where the state space was slightly larger or when policies needed to be refined over fewer iterations.

The reinforcement learning algorithms, **Q-Learning** [Watkins & Dayan \(1992\)](#) and **SARSA** [Rummery & Niranjan \(1994\)](#), demonstrated distinct characteristics in learning optimal policies through interaction with the environment. In the traffic intersection problem, Q-Learning converged to a more aggressive policy that prioritized clearing large traffic queues, while SARSA, being on-policy [Sutton & Barto \(1998\)](#), learned a more balanced approach to managing traffic. These findings highlight the flexibility of Q-Learning in solving problems with deterministic or semi-deterministic environments. On the other hand, SARSA, by considering the actual actions taken by the agent, produced policies that were more cautious, particularly in situations with unpredictable or stochastic elements.

In the warehouse robot problem, the stochastic nature of the robot's movements and the random behavior of the workers made the environment more challenging. Both Q-Learning and SARSA adapted to the uncertainty by learning policies that prioritized successful deliveries while minimizing collisions. However, Q-Learning's off-policy nature resulted in a strategy that often took more direct routes, which led to higher collision risks, whereas SARSA's on-policy updates favored safer navigation patterns. This difference underscores the importance of algorithm choice depending on whether the environment demands aggressive exploration or cautious, risk-averse behavior.

4.2 Challenges and Limitations

One of the key challenges encountered during the experiments was balancing the trade-off between exploration and exploitation, particularly in environments where stochastic transitions could significantly affect the outcomes. In both the traffic intersection and warehouse robot problems, tuning the exploration rate ϵ was critical to ensuring that the agent explored enough of the environment to learn an effective policy without compromising performance [Auer et al. \(2002\)](#). If exploration was reduced too early, the agent tended to exploit suboptimal policies, particularly in the warehouse environment where unpredictability required longer exploration periods.

Another limitation of the experiments was the computational overhead associated with **Policy Iteration** [Bellman \(1957\)](#). Although Policy Iteration converged quickly in terms of iterations, the complexity of evaluating the policy at each step, particularly in environments with larger state spaces, made it less scalable. This limitation became apparent in the warehouse problem, where the state space was more complex due to the robot's position, worker movements, and obstacle locations. In such cases, reinforcement learning algorithms like **Q-Learning** [Watkins & Dayan \(1992\)](#) and

SARSA [Rummery & Niranjana \(1994\)](#) offered more flexibility, but at the cost of requiring more episodes for convergence.

Moreover, the simplifications introduced in modeling the environments, such as grid-world representations for the warehouse robot and deterministic Poisson arrival processes for the traffic intersection, restricted the generalizability of the learned policies to more complex, real-world scenarios. For instance, in real-world traffic systems, additional factors such as weather conditions, pedestrian crossings, and vehicle types could influence decision-making. Similarly, the warehouse robot problem did not account for factors such as battery life or varying delivery priorities, which could significantly alter the optimal navigation policy in practical settings.

5 Conclusion

In this project, two distinct problems—traffic intersection control and warehouse robot navigation—were modeled as Markov Decision Processes and solved using a variety of reinforcement learning algorithms. The implementation of **Value Iteration** [Bellman \(1957\)](#), **Policy Iteration** [Bellman \(1957\)](#), **Q-Learning** [Watkins & Dayan \(1992\)](#), and **SARSA** [Rummery & Niranjana \(1994\)](#) provided valuable insights into the strengths and weaknesses of each algorithm across different environments. For the traffic intersection problem, both Value Iteration and Policy Iteration converged efficiently to optimal policies, while **Q-Learning** [Watkins & Dayan \(1992\)](#) and **SARSA** [Rummery & Niranjana \(1994\)](#) enabled flexible learning through interaction with the environment. In the warehouse robot problem, Q-Learning proved effective at maximizing deliveries, while SARSA demonstrated its strength in learning safer, more conservative policies.

Despite the successes, challenges such as balancing exploration and exploitation [Auer et al. \(2002\)](#), computational scalability, and modeling real-world complexities remained. These challenges highlight opportunities for future research, including applying deep reinforcement learning methods and improving ethical considerations in autonomous systems.

The results of this project demonstrate the potential of reinforcement learning in optimizing decision-making in dynamic environments, with significant implications for real-world applications in traffic management and warehouse automation. Future work could build upon these findings to address scalability, safety, and efficiency in more complex and practical settings.

References

- Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multi-armed bandit problem. In *Machine Learning*, volume 47, pp. 235–256. Springer, 2002.
- Richard Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957.
- Jens Kober, J. Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. In *International Journal of Robotics Research*, volume 32, pp. 1238–1274. SAGE Publications Sage UK: London, England, 2013.
- Patrick Mannion, Jim Duggan, and Enda Howley. An experimental review of reinforcement learning algorithms for adaptive traffic signal control. *Autonomous Agents and Multi-Agent Systems*, 30(6):960–983, 2016.
- Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, New Jersey, 2014.
- Gavin A. Rummery and Mahesan Niranjana. On-line q-learning using connectionist systems. In *Technical Report CUED/F-INFENG/TR 166*, Cambridge University, 1994.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 1998.

Christopher John Cornish Hellaby Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3): 279–292, 1992.